Foodulator Report
Danny Kim
Comp 373
5/4/2025

GitHub: https://github.com/FlyingTurkeySandwitch/Foodulator
Dataset (Download as csv): https://www.kaggle.com/datasets/wilmerarltstrmberg/recipe-dataset-over-2m

Abstract

Food tends to go bad faster than we expect. Whether it's leftover tomato paste from a can, a bulk purchase that's too much to finish in time, or a one-off ingredient like water chestnuts you buy for a single recipe and never touch again—these problems add up. Growing up and cooking a lot, I kept running into the same issue: once food leaves the store, it starts going stale, and without a system to use it efficiently, it often ends up wasted. My project, Foodulator, is a recipe recommender system built around this problem. A example use case for Foodulator could be: you notice certain ingredients are about to expire, you enter them into the program, and it suggests recipes you can make that use them. This not only helps cut down on food waste, but also lowers the barrier to home cooking by giving users an easy way to make use of what they already have. In the long run, I imagine something like Foodulator could be part of a smart kitchen setup—integrated with a smart fridge that detects what's inside and which ingredients are starting to go bad and recommends meals based on what needs to be used soon.

User documentation

Use Case: Searching for Recipes

1. Run Foodulator from main.py
2. Enter the ingredient you want to use in the "Enter Ingredient" field
3. Enter the tolerance (the number of ingredients you're willing to be missing from a recipe)
    a. If left blank, it defaults to 0
4. Press Submit to view matching recipes

Use Case: Adding Ingredients and Recipes*

*Recipes and ingredients must be added together to maintain data consistency due to the database's relational structure.

1. Optional Step: Run dataClean.py to generate cleaned recipes and standardized ingredient amounts
2. Steps to Insert Data Manually:
    a. Inserting Recipes:
        i. Use results from dataClean.py or a personal recipe
        ii. Format the recipe insert statement to match the example provided in the SQL file
    b. Inserting Ingredients

        i. Use results from dataClean.py or the list of ingredients used in your personal recipe
        ii. Enter each ingredient using the proper SQL insert format
        iii. Important: Ensure there are no duplicates, check the existing ingredient list first
3. Mapping Recipes to Ingredients:
    a. Use the RecipeIngredients insert statement format shown in the example.
    b. Refer to the ingredient list and output from dataClean.py to ensure accuracy.
    c. Each newly added recipe will have its recipe_id incremented by 1
    d. Excel sheet for current ingredient_ids for reference
4. After inserting all entries, rerun the program to reinitialize the database with the new data

Use Case: Using dataClean.py to Get Standardized Ingredient Amounts:

1. Open dataClean.py
2. Modify line 31 ti the number of recipes you want to clean
    a. if count == 15 (change 15 to the desired number)

3. Uncomment line 44 to print output:
    a. # print(extract_first_100_rows("recipes_data.csv"))
4. Run dataClean.py. Cleaned ingredient amounts will be printed in the terminal

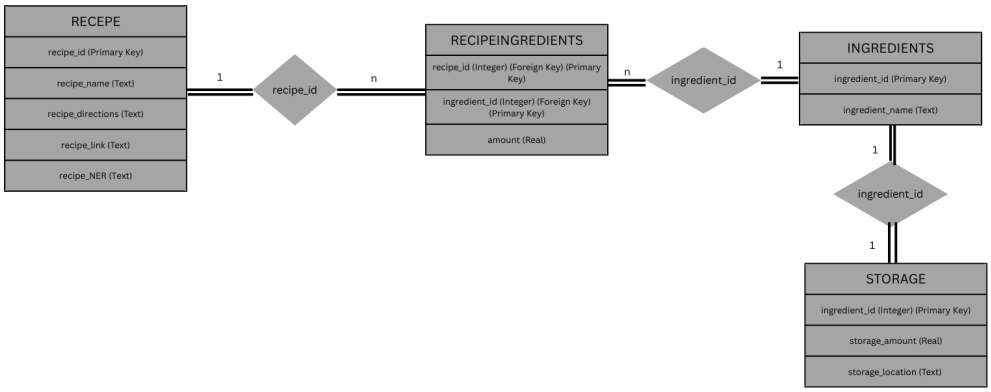
Use Case: Using dataClean.py to Get Recipes

1. Change the number of recipes by modifying the second argument on line 65
    a. print(clean_recipes("recipes_data.csv", <number>))
2. Uncomment line 65 to enable printing:
    a. # print(clean_recipes("recipes_data.csv", <number>))
3. Run dataClean.py. Cleaned recipes will be printed in the terminal

Use Case: Changing Ingredient Amount in Storage

1. Enter the name of the ingredient you want to update in the “Enter Ingredient” field
    a. (Optional) Click "Show Current Info" to view the current amount in storage
2. Enter the new amount in the “New amount” field
3. click "Update Amount" button to save the change

Engineering documentation



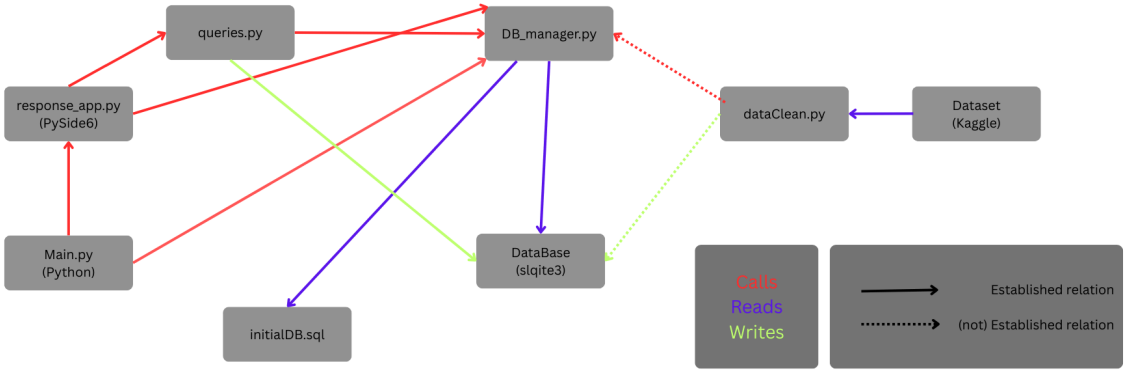
Database design:

The database was structured with normalization in mind (3NF), prioritizing data reuse, scalability, and clarity. At the core is the relationship between Recipe, RecipeIngredients, and Ingredients. A single recipe can contain many ingredients, and the same ingredient can appear in multiple recipes, so this relationship is modeled as many-to-many. To support this, RecipeIngredients acts as a join table. It contains foreign keys referencing both Recipe and Ingredients, joined by recipe_id and ingredient_id respectively. In this model, every recipe and ingredient must participate in a relationship which is represented in the ER diagram by double lines to enforce total participation. This ensures that no recipe or ingredient exists in isolation, which helps keep the dataset clean and tightly integrated. The second relationship is between Ingredients and Storage, which is a one-to-one connection. Every ingredient has a corresponding storage entry that tracks the quantity the user has on hand. This design makes it easy to update storage data directly and keep track of availability without duplicating ingredient information.

Overall, the schema is designed to reduce redundancy and support a growing recipe database. Because the join table decouples ingredients and recipes, adding new recipes with many ingredients or updating storage quantities doesn’t require any structural changes. However, a limitation in the current schema is that the amount attribute doesn't account for different units of measurement. It assumes all values are in grams, which works for many ingredients but becomes problematic for liquids or measurements that are more qualitative, like “a pinch” or “to taste.” In addition, the strict requirement that all ingredients must be used in a recipe could create problems in the future. For example, if a user adds an ingredient that hasn’t yet been linked to a recipe, which the schema doesn’t currently support storing it independently.

Small note: there are a few design choices that might seem off at first glance but were made deliberately (or out of necessity). For example, the Recipe_NER attribute in the RECIPE table stores a list, which ideally should have been split into a separate table like how ingredients are handled. But since the NER data isn't actually used in the program and was included mostly for testing and display purposes, I kept it in the main table for simplicity. On the other hand, the storage_location attribute in the STORAGE table allows for three values: fridge, freezer, and pantry. This should've been its own table as well, but that decision slipped through mostly due to time constraints and forgetfulness during the initial design.

Program Design:



This project was built with modularity in mind. Different components were separated to make debugging and future development easier. The main entry point is main.py, which initializes the database and launches the GUI. Currently, the database is reinitialized every time the program runs. This makes it easy to test, especially during development, but would need to be changed for real life applications.

Database operations are handled through db_manager.py, which centralizes all connection logic. SQL queries themselves are separated out into queries.py, which allows for easier testing and future database migrations if needed. The GUI, built using PySide6, lives in response_app.py, which handles user input and output. The GUI directly calls query functions and formats the returned data before displaying it. While this structure works for now, it results in some tight coupling between the GUI and database logic. Ideally, an additional middle layer would handle that interaction to keep things cleaner.

There's also a file called dataClean.py, which is designed to help with importing new recipe datasets. It's not fully integrated yet but was created with future scalability in mind. The

idea is that users could upload new recipes, and the script would clean the data before inserting it into the database. At the moment, this cleaned data still has to be manually transferred into initialDB.sql, which defines the schema and populates the database. That SQL file ensures reproducibility and is the main place where demo data is stored and changed.

In terms of maintainability, the project is straightforward to update. To change the database schema or the demo content, you only need to edit initialDB.sql and rerun the program. A virtual environment was used to isolate dependencies, mainly because PySide6 recommends one, and it helps avoid version conflicts with other Python projects.

Limitations:

There are several trade-offs in the current design. For starters, the program uses raw SQL strings throughout. While this makes the code easy to write and understand, it introduces potential security risks, especially if future versions accept user input. Switching to parameterized queries or an ORM would help with that. The database is also local and small-scale, which limits its usefulness in multi-user or web-based environments. Expanding the project to support multiple users or online access would require a move to a more robust database system like PostgreSQL.

The GUI itself is functional but basic. PySide6 is solid for simple applications but isn't the most visually flexible framework. There's no authentication system, no error handling for invalid inputs, and no easy way for users to update information in the database unless they know how to modify the SQL directly. While modularity was a goal, the current structure relies on implicit boundaries. For example, response_app.py does more than just display information—it also calls SQL functions and processes their outputs. That means future changes, like tracking nutritional information or adding filtering logic, would require changes in multiple places.

Setup:

1. Clone repository
2. Set up a virtual environment
    a. Technically, I don't think it's needed, but PySide6 documentation recommends it
3. Dependency details are located in requirements.txt, and would need to be installed
    a. Key libraries: PySide6, Pandas, sqlite3
    b. If using dataClean.py, need to update API_KEY (line 11) in dataClean.py
4. Run the program from main.py

Proactive Planning:

There are a few known limitations in the current implementation that were, unfortunately, left unresolved, mostly due to time constraints and project scope. The main unresolved limitation is that users can't add new ingredients or recipes directly through the GUI. Any updates to the recipe or ingredient list must be made manually by editing initialDB.sql and rerunning the program. This obviously isn't ideal for long-term usability, but it made development

simpler during the initial phase. The dataClean.py script does function correctly and is capable of processing raw recipe data, but it doesn't automatically insert that data into the database. For now, any cleaned data still needs to be transferred manually into the SQL schema which requires data-entry labor and assumes user is proficient in SQL.

Looking ahead, one of the main priorities will be automating the process of adding new ingredients and recipes into the database. While users still have to manually insert cleaned data through initialDB.sql, the dataClean.py script already uses an LLM API to process raw recipe inputs. The script is timed to avoid exceeding token limits, which helps prevent interruptions during cleaning. I made early design decisions to support long-term development: using common libraries (to my knowledge) like PySide6, sqlite3, and pandas, and keeping the GUI and database logic separated. This structure should make it easier to test and integrate future features like automatic uploads or user-facing data entry through the interface.

Engineering retrospective

Overall, I think the planning phase of the project went pretty well. Having a structured project plan (from HW5) really helped me map out the different components I needed to build. It gave me a clearer picture of how the pieces would eventually connect, especially early on when things were still abstract. Knowing what the cleaned data was going to look like, even though I had to adjust the format a few times, also gave me a solid starting point. Just having a general idea of what "good" data should look like made development more manageable.

That said, time management was still a challenge. I've improved compared to past projects, but I found myself slipping into the same pattern of doing intense bursts of work right before deadlines and then stepping away from the project for a week or two. Part of this was because I was working solo, which was a very different experience from group projects I've done in other classes. Without set meeting times or teammates to stay accountable to, it was easy to deprioritize the project until it became urgent because everything was based on my schedule. I also underestimated how much slower solo work would be. I'm used to getting assigned a specific task in a group and knocking it out in a day or two—but in this project, I had to handle everything: frontend, backend, design decisions, data cleaning, and testing. I found that while I could still finish a small task (like writing a function) in a day, estimating how many of those I could get through in a week was way off, mostly because of other responsibilities and classwork piling up but also because this was just a very new experience for me.

I intentionally kept the project small to stay within scope, but when I eventually wanted to expand it, that was harder than I expected. Even though I designed the system to be modular, it turns out modularity on paper doesn't always translate to easy extensibility in practice. For example, if I wanted to add nutritional data, it wouldn't just be a new output field, it would require updated data cleaning, new logic for full and partial ingredient matches, extra SQL queries, and another display method. In hindsight, I would have needed to break components down even further to make those kinds of extensions easier.

Tracking how much time I spent on the project is hard because my work pattern was uneven. If I had to estimate, I'd say I averaged about 10 hours per week overall, with a big chunk of that time going into writing and debugging SQL statements, since that was one of the more finicky and detail-oriented parts of the system.